

Material Sorting Task

运行环境版本

- CUDA: 12.8 (Docker 基础镜像为 `nvcr.io/nvidia/cuda:12.8.0-devel-ubuntu22.04`)
- PyTorch: 2.8.0+cu128
- torchvision: 0.23.0+cu128

文旅机器人物料分拣 ROS2 仿真环境。Server 运行 MuJoCo + 可选 3DGS 场景，发布相机、深度、里程计和关节状态；参赛程序通过 ROS2 话题控制 MMK2 移动双臂机器人识别、抓取并搬运彩色长方体物料箱。

部署

宿主机需要 Docker、NVIDIA Driver、NVIDIA Container Toolkit 和 NVIDIA GPU。下载离线镜像包：

- Server 镜像包：[链接: <https://pan.baidu.com/s/1JaRLqUEzzmCNiOCensxoHQ> 提取码: mxts]
- Client 镜像包：[链接: <https://pan.baidu.com/s/1wpUAAMnPNpYCUf9movmYAA> 提取码: atvs]

下载后导入镜像：

```
docker load -i material_sorting_server.tar
docker load -i material_sorting_client.tar
```

允许运行窗口显示，并创建 gsplat 编译缓存：

```
xhost +local:docker
docker volume create material_sorting_cache_cu128
```

随机规则

每次启动 server 默认开启随机布局，三个可抓取彩色箱子 pink、yellow、brown 会随机分配到以下三个槽位：

槽位	随机规则	姿态
桌面侧边	白色正方体障碍物左侧或右侧随机出现 1 个箱子	横放，长边沿桌面左右方向
桌面顶部	白色正方体障碍物顶部随机出现 1 个箱子	竖放，长边沿桌面前后方向
货架	从下往上数第 1/2/3 层随机出现 1 个箱子	横放，长边沿货架横向

白色正方体障碍物固定在桌面。白色长方体障碍物固定不可移动，随机出现在货架中与彩色箱子不同的另一层；剩余层为空层，用作任务一放置目标层。

可用 `MATERIAL_SEED=<整数>` 固定随机种子；不设置时 server 自动生成并打印 seed。设置 `MATERIAL_RANDOMIZE=0` 可使用固定布局。

固定 Baseline

这一组命令用于验证固定场景任务一 baseline。`client_task_1.py` 针对固定调试场景手工调参，不适配正式随机布局，因此需要关闭随机。

Server

```
docker run --rm -it \  
  --gpus all \  
  --network host \  
  --ipc host \  
  --name material_sorting_server \  
  -e DISPLAY=${DISPLAY} \  
  -e ROS_DOMAIN_ID=99 \  
  -e RMW_IMPLEMENTATION=rmw_cyclonedds_cpp \  
  -e MUJOCO_GL=glfw \  
  -e MATERIAL_ENABLE_RENDER=1 \  
  -e MATERIAL_USE_GS=1 \  
  -e MATERIAL_RANDOMIZE=0 \  
  -e MATERIAL_ENABLE_SCORE=1 \  
  -v /tmp/.X11-unix:/tmp/.X11-unix:rw \  
  -v material_sorting_cache_cu128:/opt/torch_ext \  
  material_sorting:offline-server \  
  python3 examples/material_sorting/material_sorting_server.py
```

Client

在 baseline 仓库根目录另开一个终端，启动固定 Client 容器：

```
docker run --rm -dit \  
  --gpus all \  
  --network host \  
  --ipc host \  
  --name material_sorting_client \  
  -e ROS_DOMAIN_ID=99 \  
  -e RMW_IMPLEMENTATION=rmw_cyclonedds_cpp \  
  -v "$PWD":/workspace/baseline:ro \  
  material_sorting:offline-client
```

Baseline

在 Client 容器中启动 YOLO 感知节点和任务一 baseline：

```
docker exec -it material_sorting_client bash -lc '  
cd /workspace/baseline/examples/material_sorting  
python3 perception/box_detect.py --backend yolo &  
det_pid=$!  
trap "kill $det_pid 2>/dev/null || true" EXIT  
python3 client_task_1.py  
,
```

正式 Server

正式评测使用随机布局并开启裁判。每次启动都会重新随机彩色箱子槽位、桌面侧边左右、货架箱子层数和长方体障碍物层数。正式运行默认不设置 `MATERIAL_SEED`；需要复现某次布局时再手动添加。

```
docker run --rm -it \  
--gpus all \  
--network host \  
--ipc host \  
--name material_sorting_server \  
-e DISPLAY=${DISPLAY} \  
-e ROS_DOMAIN_ID=99 \  
-e RMW_IMPLEMENTATION=rmw_cyclonedds_cpp \  
-e MUJOCO_GL=glfw \  
-e MATERIAL_ENABLE_RENDER=1 \  
-e MATERIAL_USE_GS=1 \  
-e MATERIAL_RANDOMIZE=1 \  
-e MATERIAL_ENABLE_SCORE=1 \  
-v /tmp/.X11-unix:/tmp/.X11-unix:rw \  
-v material_sorting_cache_cu128:/opt/torch_ext \  
material_sorting:offline-server \  
python3 examples/material_sorting/material_sorting_server.py
```

Client

在选手程序仓库根目录另开一个终端，启动固定 Client 容器：

```
docker run --rm -dit \
  --gpus all \
  --network host \
  --ipc host \
  --name material_sorting_client \
  -e ROS_DOMAIN_ID=99 \
  -e RMW_IMPLEMENTATION=rmw_cyclonedds_cpp \
  -v "$PWD":/workspace/baseline:ro \
  material_sorting:offline-client
```

然后在 Client 容器中启动选手程序：

```
docker exec -it material_sorting_client bash -lc '
  cd /workspace/baseline
  python3 your_client.py
'
```

任务指令

server 会在 `/material/instruction` 发布 JSON 指令列表。随机布局下指令按槽位生成：

1. 抓取桌面左/右侧的彩色箱子，放到货架空层。
2. 抓取货架中的彩色箱子，放到第一个箱子原来在桌面侧边的位置。
3. 抓取白色正方体顶部的彩色箱子，放到货架中白色长方体障碍物的左边。

每条指令包含 `target_color`、`target_body`、`place_world`、`place_type` 等字段。正式参赛程序应读取 `/material/instruction`，结合视觉检测结果判断目标颜色和当前位置，不要硬编码固定 baseline 的 pink/brown/yellow 顺序。

评分标准

server 内置裁判，用 MuJoCo 地面真值实时判分。开启 `MATERIAL_ENABLE_SCORE=1` 后发布：

Topic	Type	说明
/referee/taskinfo	std_msgs/msg/String	当前任务和指令信息
/referee/gameinfo	std_msgs/msg/String	实时进度、分数、尝试次数

Topic	Type	说明
/referee/score	std_msgs/msg/Int32	当前总分

每个任务最多 3 次机会，按最高分计入总分。规则阈值在 Server 镜像内配置；退出 server 时会输出 referee_results_<timestamp>.json。

ROS2 话题

ROS_DOMAIN_ID 必须在 server 和参赛程序之间保持一致。下表只列比赛相关接口。

Server 发布

Topic	Type	说明
/slamware_ros_sdk_server_node/odom	nav_msgs/msg/Odometry	底盘世界位姿和速度
/tf	tf2_msgs/msg/TFMessage	动态 TF，主要为 odom -> base_link
/joint_states	sensor_msgs/msg/JointState	升降、头部、双臂和夹爪关节状态
/head_camera/color/image_raw	sensor_msgs/msg/Image	头部 RGB 图像，默认 640x480
/head_camera/color/camera_info	sensor_msgs/msg/CameraInfo	头部 RGB 相机内参
/head_camera/aligned_depth_to_color/image_raw	sensor_msgs/msg/Image	头部深度图，mono16，单位毫米
/head_camera/aligned_depth_to_color/camera_info	sensor_msgs/msg/CameraInfo	头部深度相机内参
/left_camera/color/image_raw	sensor_msgs/msg/Image	左腕 RGB 图像
/left_camera/color/camera_info	sensor_msgs/msg/CameraInfo	左腕相机内参
/right_camera/color/image_raw	sensor_msgs/msg/Image	右腕 RGB 图像
/right_camera/color/camera_info	sensor_msgs/msg/CameraInfo	右腕相机内参
/material/instruction	std_msgs/msg/String	任务指令 JSON

Server 订阅

Topic	Type	控制格式
/cmd_vel	geometry_msgs/msg/Twist	linear.x 前进速度，angular.z 角速度
/spine_forward_position_controller/commands	std_msgs/msg/Float64MultiArray	[slide_joint]
/head_forward_position_controller/commands	std_msgs/msg/Float64MultiArray	[head_yaw_joint, head_pitch_joint]

Topic	Type	控制格式
/left_arm_forward_position_controller/commands	std_msgs/msg/Float64MultiArray	[joint1..joint6, gripper]
/right_arm_forward_position_controller/commands	std_msgs/msg/Float64MultiArray	[joint1..joint6, gripper]

Baseline 感知节点发布

Topic	Type	说明
/material/detections	vision_msgs/msg/Detection3DArray	彩色箱检测结果，世界坐标系，class_id=pink/yellow/brown
/material/result_image	sensor_msgs/msg/Image	检测可视化图，仅调试用

参数说明

参数	推荐值	含义
ROS_DOMAIN_ID	99	ROS2 通信域，server/client 必须一致
RMW_IMPLEMENTATION	rmw_cyclonedds_cpp	ROS2 RMW 实现
MUJOCO_GL	glfw	图形窗口渲染
MATERIAL_ENABLE_RENDER	1	发布 RGB-D 相机数据
MATERIAL_USE_GS	1	启用 3D Gaussian Splatting 渲染
MATERIAL_RANDOMIZE	1	是否开启随机布局；正式 server 默认开启
MATERIAL_SEED	自动生成，或指定整数	固定随机种子，使布局可复现
MATERIAL_ENABLE_SCORE	1	启用裁判和 /referee/* 话题

主要文件

examples/material_sorting/client_task_1.py	# 固定场景任务一 baseline
examples/material_sorting/perception/box_detect.py	# YOLO 检测节点
examples/material_sorting/perception/backends.py	# 检测后端
examples/material_sorting/perception/checkpoints/material_box.pt	# YOLO 权重
examples/material_sorting/models/	# FK 所需机器人模型资源

视觉检测

感知节点使用 YOLO 后端：

```
yolo 训 练 好 的 YOLO 权 重  
examples/material_sorting/perception/checkpoints/material_box.pt
```

启动：

```
cd examples/material_sorting  
python3 perception/box_detect.py --backend yolo
```

YOLO 类 别 顺 序 固 定 为 [pink, yellow, brown] ， 与
perception/backends.py::YoloBackend.CLASS_NAMES 一致。